

Dynamic Programming: Reinforcement Learning - Lecture 3

Felipe Meneguzzi
`felipe.meneguzzi@pucrs.br`

PUCRS

Material adapted from:
Sebastian Thrun - Stanford University/Udacity
Sutton and Barto - Reinforcement Learning: An Introduction (2nd ed.)
David Silver - Reinforcement Learning

Outline

- ① Recap
- ② Introduction
- ③ Policy Evaluation
- ④ Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- ⑤ Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- ⑥ Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- ⑦ Contraction Mapping

- Last lecture
 - Markov Decision Processes
 - Value functions
- This lecture (Sutton and Barto 4.1-4.8)
 - **Planning by dynamic programming**
 - Solving a **known** MDP
- Next Lecture
 - Bandit problems
 - Exploration and exploitation in single decisions

Recap

- We understand underlying formalisms for discounted reward, stochastic Planning Problems
 - Markov Reward Processes
 - Markov Decision Processes
- We know how to calculate immediate rewards
- But, how can we get the agent to decide between different long-term courses of action, in the face of uncertainty?

- **Markov Decision Processes (MDPs)** formally describe environments for reinforcement learning
- Assumptions behind MDP environments
 - Fully observable states (states completely characterizes the process)
 - Stochastic transitions
 - Agent has **partial control** over transitions
- Almost all RL problems can be formalized as MDPs, e.g.:
 - Optimal control primarily deals with continuous MDPs
 - Partially observable problems can be converted into MDPs
 - Bandits are MDPs with one state

Policies Quiz

	1	2	3	4
a				+100
b				-100
c				

What is the optimal action when you are in:
a1: N S W E

Policies Quiz

	1	2	3	4
a				+100
b				-100
c				

What is the optimal action when you are in:

a1: N S W E $\rightarrow$ E

c1: N S W E

Policies Quiz

	1	2	3	4
a				+100
b				-100
c				

What is the optimal action when you are in:

a1: N S W E $\rightarrow$ E

c1: N S W E $\rightarrow$ N

c4: N S W E

Policies Quiz

	1	2	3	4
a				+100
b				-100
c				

What is the optimal action when you are in:

a1: N S W E $\rightarrow$ E

c1: N S W E $\rightarrow$ N

c4: N S W E $\rightarrow$ S

b3: N S W E

Policies Quiz

	1	2	3	4
a				+100
b				-100
c				

This is somewhat unintuitive, why?

What is the optimal action when you are in:

a1: N S W E $\rightarrow$ E

c1: N S W E $\rightarrow$ N

c4: N S W E $\rightarrow$ S

b3: N S W E $\rightarrow$ W

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\sum_{t=0}^{\infty} R_t$$

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\mathbb{E} \left[\sum_{t=0}^{\infty} R_t \right]$$

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\mathbb{E} \left[\sum_{t=0}^{\infty} R_t \right] \rightarrow \max$$

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \rightarrow \max$$

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \rightarrow \max$$

γ = discount factor

MDPs and Costs

	1	2	3	4
a				+100
b				-100
c				

$$R(s) = \begin{cases} +100 & a4 \\ -100 & b4 \\ -3 & \text{other states} \end{cases}$$

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \rightarrow \max$$

γ = discount factor $\gamma = 0.9$

Value Functions

	1	2	3	4
a	-3	-3	-3	+100
b	-3		-3	-100
c	-3	-3	-3	-3

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t) | s_0 = s \right]$$

Value Functions

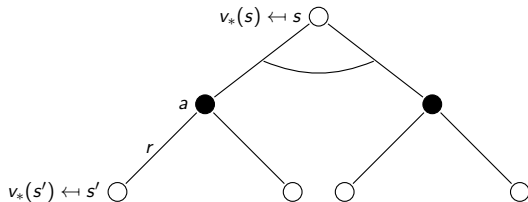
	1	2	3	4
a	-3	-3	-3	+100
b	-3		-3	-100
c	-3	-3	-3	-3

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t) | s_0 = s \right]$$

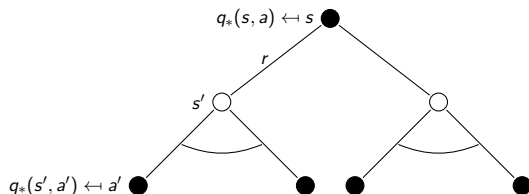
Planning = Calculate Value Functions

Recap

We have seen the relation between the various formulations of an MDP's value functions



$$v_*(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right]$$



$$q_*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s', a')$$

How do we use these functions to create planning algorithms?

Outline

- 1 Recap
- 2 Introduction
- 3 Policy Evaluation
- 4 Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- 5 Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- 6 Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- 7 Contraction Mapping

What is Dynamic Programming?

Dynamic sequential or temporal component to the problem

Programming optimising a “program”, i.e. a policy
c.f. linear programming

- A method for solving complex problems
- By breaking them down into subproblems
 - Solve the subproblems
 - Combine solutions to subproblems

Requirements for Dynamic Programming

- Optimal substructure
 - Principle of optimality applies
 - Optimal solution can be decomposed into subproblems
- Overlapping subproblems
 - Subproblems recur many times
 - Solutions can be cached and reused
- Markov decision processes satisfy both properties
 - Bellman equation gives recursive decomposition
 - Value function stores and reuses solutions

Planning by Dynamic Programming

- Dynamic programming assumes full knowledge of the MDP
- It is used for **planning** in an MDP
- For prediction:
 - Input: MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ and policy π
or: MRP $\langle \mathcal{S}, \mathcal{P}^\pi, \mathcal{R}^\pi, \gamma \rangle$
 - Output: value function v_π
- Or for control:
 - Input: MDP $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$
 - Output: optimal value function v_*
and: optimal policy π_*

Other Applications of Dynamic Programming

Dynamic programming is used to solve many other problems, e.g.

- Scheduling algorithms
- String algorithms (e.g. sequence alignment)
- Graph algorithms (e.g. shortest path algorithms)
- Graphical models (e.g. Viterbi algorithm)
- Bioinformatics (e.g. lattice models)

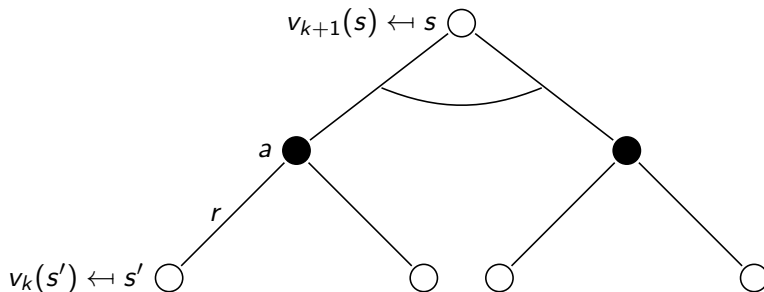
Outline

- 1 Recap
- 2 Introduction
- 3 Policy Evaluation**
- 4 Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- 5 Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- 6 Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- 7 Contraction Mapping

Iterative Policy Evaluation

- Problem: evaluate a given policy π
- Solution: iterative application of Bellman expectation backup
- $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$
- Using synchronous backups,
 - At each iteration $k + 1$
 - For all states $s \in \mathcal{S}$
 - Update $v_{k+1}(s)$ from $v_k(s')$
 - where s' is a successor state of s
- **asynchronous** backups later
- Provable convergence to v_*

Iterative Policy Evaluation

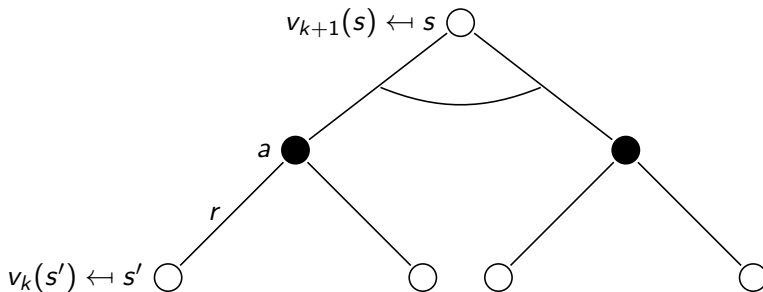


For deterministic policies

$$v_{k+1}(s) = \mathcal{R}_s^{\pi(s)} + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^{\pi(s)} v_k(s')$$

$$\mathbf{v}^{k+1} = \mathcal{R}^{\pi} + \gamma \mathcal{P}^{\pi} \mathbf{v}^k$$

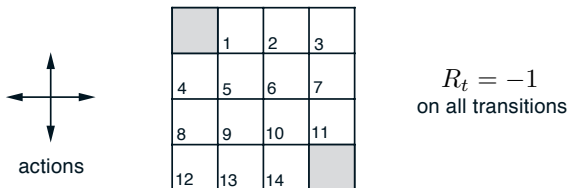
Iterative Policy Evaluation



For stochastic policies

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_k(s') \right)$$
$$\mathbf{v}^{k+1} = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi \mathbf{v}^k$$

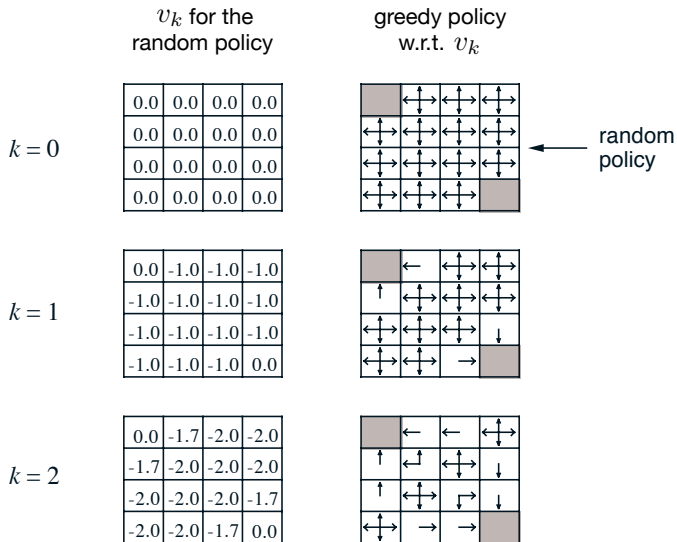
Evaluating a Random Policy in the Small Gridworld



- Undiscounted episodic MDP ($\gamma = 1$)
- Nonterminal states $1, \dots, 14$
- One terminal state (shown twice as shaded squares)
- Actions leading out of the grid leave state unchanged
- Reward is -1 until the terminal state is reached
- Agent follows uniform random policy

$$\pi(n \mid \cdot) = \pi(e \mid \cdot) = \pi(s \mid \cdot) = \pi(s \mid \cdot) = 0.25$$

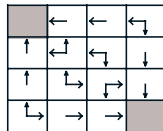
Iterative Policy Evaluation in Small Gridworld



Iterative Policy Evaluation in Small Gridworld (2)

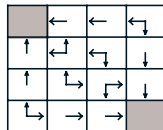
$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0



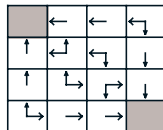
$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0



$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0



optimal policy

Outline

- 1 Recap
- 2 Introduction
- 3 Policy Evaluation
- 4 Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- 5 Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- 6 Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- 7 Contraction Mapping

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi' = \text{greedy}(v_{\pi})$$

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} \sum_{s'} \mathcal{P}_{ss'}^a v_{\pi}(s')$$

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi'(a \mid s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} \sum_{s'} \mathcal{P}_{ss'}^a v_{\pi}(s') \\ 0 & \text{otherwise} \end{cases}$$

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi'(a \mid s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} \sum_{s'} \mathcal{P}_{ss'}^a v_{\pi}(s') \\ 0 & \text{otherwise} \end{cases}$$

- In Small Gridworld improved policy was optimal, $\pi' = \pi_*$

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi'(a \mid s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} \sum_{s'} \mathcal{P}_{ss'}^a v_{\pi}(s') \\ 0 & \text{otherwise} \end{cases}$$

- In Small Gridworld improved policy was optimal, $\pi' = \pi_*$
- In general, need more iterations of improvement / evaluation

How to Improve a Policy

- Given a policy π
 - **Evaluate** the policy π

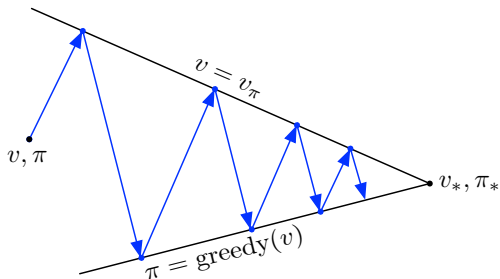
$$v_{\pi}(s) = \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s]$$

- **Improve** the policy by acting greedily with respect to v_{π}

$$\pi'(a \mid s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} \sum_{s'} \mathcal{P}_{ss'}^a v_{\pi}(s') \\ 0 & \text{otherwise} \end{cases}$$

- In Small Gridworld improved policy was optimal, $\pi' = \pi_*$
- In general, need more iterations of improvement / evaluation
- But this process of **policy iteration** always converges to π_*

Policy Iteration

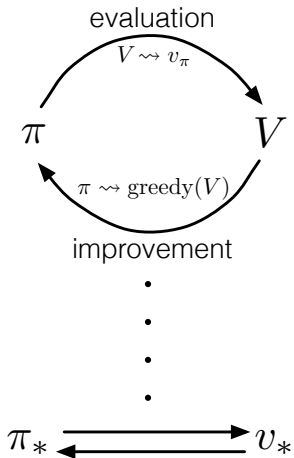


Policy evaluation Estimate v_π

Iterative policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Greedy policy improvement

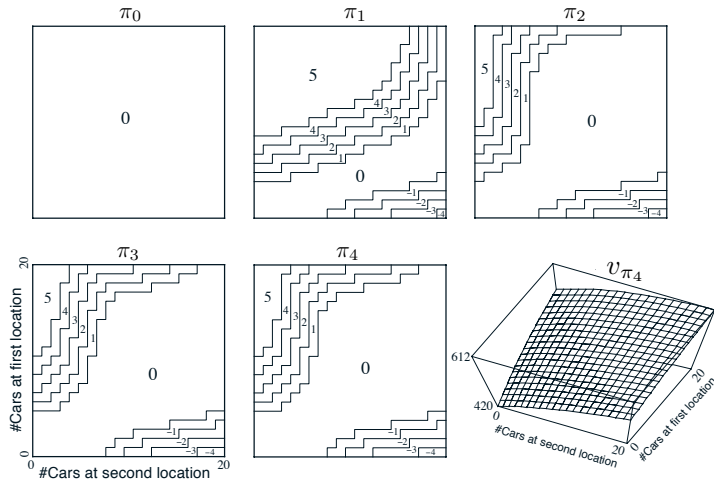


Jack's Car Rental



- States: Two locations, maximum of 20 cars at each
- Actions: Move up to 5 cars between locations overnight
- Reward: \$10 for each car rented (must be available)
- Transitions: Cars returned and requested randomly
 - Poisson distribution, n returns/requests with prob $\frac{\lambda^n}{n!}e^{-\lambda}$
 - 1st location: average requests = 3, average returns = 3
 - 2nd location: average requests = 4, average returns = 2

Policy Iteration in Jack's Car Rental



Policy Improvement

- Consider a deterministic policy, $a = \pi(s)$

Policy Improvement

- Consider a deterministic policy, $a = \pi(s)$
- We can **improve** the policy by acting greedily

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

Policy Improvement

- Consider a deterministic policy, $a = \pi(s)$
- We can **improve** the policy by acting greedily

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

- This improves the value from any state s over one step,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

Policy Improvement

- Consider a deterministic policy, $a = \pi(s)$
- We can **improve** the policy by acting greedily

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

- This improves the value from any state s over one step,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) \geq q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- It therefore improves the value function, $v_{\pi'}(s) \geq v_{\pi}(s)$

$$\begin{aligned} v_{\pi}(s) &\leq q_{\pi}(s, \pi'(s)) = \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma q_{\pi}(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma S_{t+2} + \gamma^2 q_{\pi}(S_{t+2}, \pi'(S_{t+2})) \mid S_t = s] \\ &\leq \mathbb{E} [R_{t+1} + \gamma R_{t+2} + \dots \mid S_t = s] = v_{\pi'}(s) \end{aligned}$$

Policy Improvement (2)

- If improvement stops,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

Policy Improvement (2)

- If improvement stops,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- Then the Bellman optimality equation has been satisfied

$$v_{\pi}(s) = \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

Policy Improvement (2)

- If improvement stops,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- Then the Bellman optimality equation has been satisfied

$$v_{\pi}(s) = \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

- Therefore $v_{\pi}(s) = v_{*}(s)$ for all $s \in \mathcal{S}$

Policy Improvement (2)

- If improvement stops,

$$q_{\pi}(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_{\pi}(s, a) = q_{\pi}(s, \pi(s)) = v_{\pi}(s)$$

- Then the Bellman optimality equation has been satisfied

$$v_{\pi}(s) = \max_{a \in \mathcal{A}} q_{\pi}(s, a)$$

- Therefore $v_{\pi}(s) = v_{*}(s)$ for all $s \in \mathcal{S}$
- So π is an optimal policy

Policy Iteration Algorithm

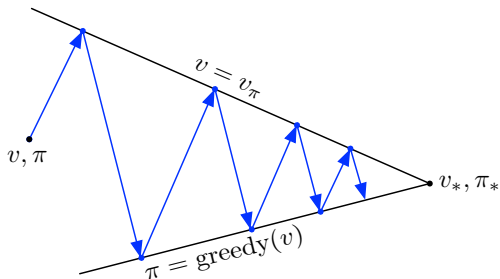
Require: mdp , an MDP with states $\mathcal{S}$, actions $\mathcal{A}$, and transition model $\mathcal{P}$

```
1: function PolicyIteration( $mdp$ )
2:    $v \leftarrow 0$  for each  $s \in \mathcal{S}$ 
3:    $\pi \leftarrow$  random action  $a \in \mathcal{A}$  for each  $s \in \mathcal{S}$ 
4:   repeat
5:      $v \leftarrow$  PolicyEvaluation( $\pi, v, mdp$ )
6:      $unchanged \leftarrow true$ 
7:     for each state  $s \in \mathcal{S}$  do
8:       if  $\max_{a \in \mathcal{A}(s)} \sum_{s'} \mathcal{P}(s' | s, a) v[s'] > \sum_{s'} \mathcal{P}(s' | s, \pi[s]) v[s']$  then
9:          $\pi[s] \leftarrow \arg \max_{a \in \mathcal{A}(s)} \sum_{s'} \mathcal{P}(s' | s, a) v[s']$ 
10:       $unchanged \leftarrow false$ 
11:   until  $unchanged$ 
12:   return  $v$ 
```

Modified Policy Iteration

- Does policy evaluation need to converge to v_π ?
- Or should we introduce a stopping condition
 - e.g. ϵ -convergence of value function
- Or simply stop after k iterations of iterative policy evaluation?
- For example, in the small gridworld $k = 3$ was sufficient to achieve optimal policy
- Why not update policy every iteration? i.e. stop after $k = 1$
 - This is equivalent to value iteration

Generalized Policy Iteration

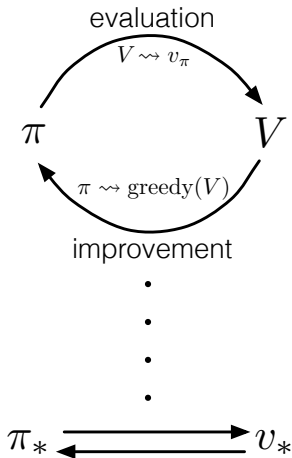


Policy evaluation Estimate v_π

Any policy evaluation

Policy improvement Generate $\pi' \geq \pi$

Any policy improvement algorithm

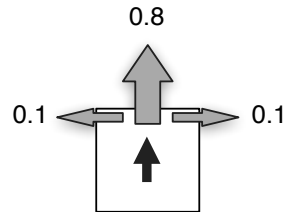


Outline

- ① Recap
- ② Introduction
- ③ Policy Evaluation
- ④ Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- ⑤ Value Iteration**
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- ⑥ Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- ⑦ Contraction Mapping

Example

	1	2	3	4
a				+100
b				-100
c	START			



Bellman Equation for Deterministic Policies

	1	2	3	4
a	0	0	0	+100
b	0		0	-100
c	0	0	0	0

$$q(a3, E) = ?$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	0	0	0	+100
b	0		0	-100
c	0	0	0	0

$$q(a3, E) = ?$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	0	0	77	+100
b	0		0	-100
c	0	0	0	0

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \mathcal{R}_s$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \mathcal{R}_s + v(s')$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \mathcal{R}_s + \left(\sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \mathcal{R}_s + \left(\gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

Bellman Equation for Deterministic Policies

	1	2	3	4
a	85	89	93	+100
b	81		68	-100
c	77	73	70	47

$$q(a3, E) = -3 + 0.8 * 100 = 77$$

$$v(s) \leftarrow \begin{cases} \mathcal{R}_s & \text{if terminal} \\ \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right) \end{cases}$$

Back up function

When $v(s) = \mathcal{R}_s + (\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s'))$ (converges)
it becomes the **Bellman Equation**

Principle of Optimality

Any optimal policy can be subdivided into two components:

- An optimal first action A_*
- Followed by an optimal policy from successor state S'

Theorem

A policy $\pi(a | s)$ achieves the optimal value from state s , $v_\pi(s) = v_(s)$, if and only if*

- *For any state s' reachable from s*
- *π achieves the optimal value from state s' , $v_\pi(s') = v_*(s')$*

Deterministic Value Iteration

- If we know the solution to subproblems $v_*(s')$
- Then solution $v_*(s)$ can be found by one-step lookahead

$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right)$$

- The idea of value iteration is to apply these updates iteratively
- Intuition: start with final rewards and work backwards
- Still works with loopy, stochastic MDPs

Example: Shortest Path

g			

Problem

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

V_1

0	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1
-1	-1	-1	-1

V_2

0	-1	-2	-2
-1	-2	-2	-2
-2	-2	-2	-2
-2	-2	-2	-2

V_3

0	-1	-2	-3
-1	-2	-3	-3
-2	-3	-3	-3
-3	-3	-3	-3

V_4

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-4
-3	-4	-4	-4

V_5

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-5

V_6

0	-1	-2	-3
-1	-2	-3	-4
-2	-3	-4	-5
-3	-4	-5	-6

V_7

Value Iteration

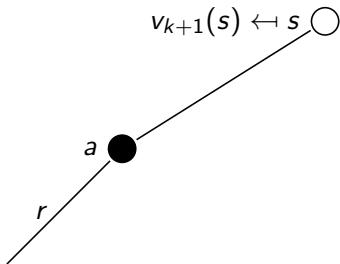
- Problem: find optimal policy π_*
- Solution: iterative application of Bellman optimality backup
- $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$
- Using synchronous backups
 - At each iteration $k + 1$
 - For all states $s \in \mathcal{S}$
 - Update $v_{k+1}(s)$ from $v_k(s')$
- Provably converges to π_*
- Unlike policy iteration, there is no explicit policy
- Intermediate value functions may not correspond to any policy

Value Iteration (2)

$$v_{k+1}(s) \leftarrow s \bigcirc$$

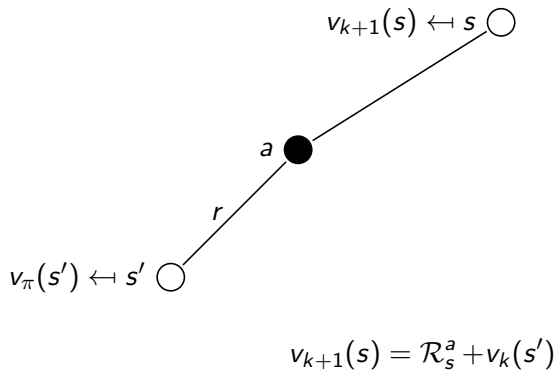
$$v_{k+1}(s) =$$

Value Iteration (2)

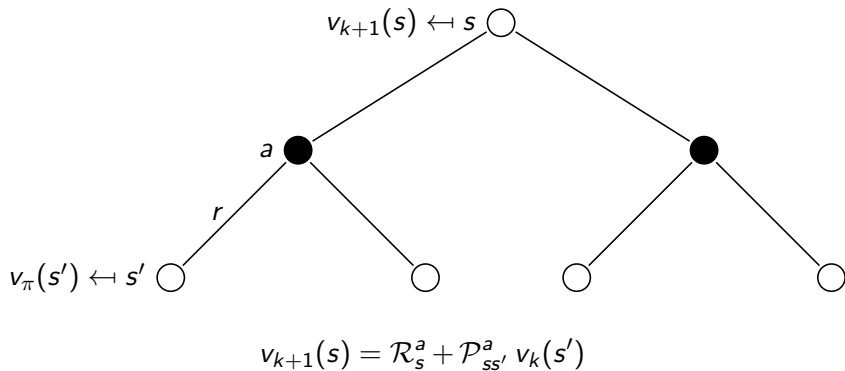


$$v_{k+1}(s) = \mathcal{R}_s^a$$

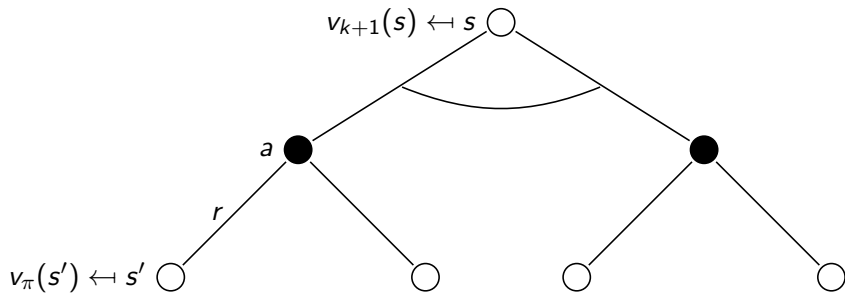
Value Iteration (2)



Value Iteration (2)



Value Iteration (2)



$$v_{k+1}(s) = \max_a \left(\mathcal{R}_s^a + \gamma \sum_{s'} \mathcal{P}_{ss'}^a v_k(s') \right)$$

Value Iteration - Deterministic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$R(s) = -3$ and $\gamma = 1$

What are the values of the following states, given a **deterministic** T ? $v(a3) = ?$

Value Iteration - Deterministic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$R(s) = -3$ and $\gamma = 1$

What are the values of the following states, given a **deterministic** T ? $v(a3) = ?$

Value Iteration - Deterministic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$R(s) = -3$ and $\gamma = 1$

What are the values of the following states, given a **deterministic** T ? $v(a3) = 97$
 $v(b3) = ?$

Value Iteration - Deterministic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$R(s) = -3$ and $\gamma = 1$

What are the values of the following states, given a **deterministic** T ? $v(a3) = 97$

$v(b3) = 94$

$v(c1) = ?$

Value Iteration - Deterministic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a	91	94	97	+100
b	88		94	-100
c	85	88	91	

$R(s) = -3$ and $\gamma = 1$

What are the values of the following states, given a **deterministic** T ? $v(a3) = 97$

$v(b3) = 94$

$v(c1) = 85$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$\mathcal{R}_s = -3$ and $\gamma = 1$

What are the values of the states below, given:

- **stochastic** T , $p = 0.8$
- $v(s)$ originally 0

$v(a3) = ?$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a				+100
b				-100
c				

$\mathcal{R}_s = -3$ and $\gamma = 1$

What are the values of the states below, given:

- **stochastic** T , $p = 0.8$
- $v(s)$ originally 0

$v(a3) = ?$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a			77	+100
b				-100
c				

$\mathcal{R}_s = -3$ and $\gamma = 1$

What are the values of the states below, given:

- stochastic T , $p = 0.8$
- $v(s)$ originally 0

$v(a3) = 77$

$v(b3) = ?$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a			77	+100
b				-100
c				

$\mathcal{R}_s = -3$ and $\gamma = 1$

What are the values of the states below, given:

- stochastic T , $p = 0.8$
- $v(s)$ originally 0

$$v(a3) = 77$$

$$v(b3) = 48.6$$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a			77	+100
b				-100
c				

What are the values of the states below, given:

- stochastic T , $p = 0.8$
- $v(s)$ originally 0

$$v(a3) = 77$$

$$v(b3) = 48.6$$

$$\mathcal{R}_s = -3 \text{ and } \gamma = 1$$

$$\text{N: } -3 + [0.8 * 77 + 0.1 * (-100) + 0.1 * 0] = 48.6$$

Value Iteration - Stochastic Quiz

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

	1	2	3	4
a			77	+100
b				-100
c				

What are the values of the states below, given:

- stochastic T , $p = 0.8$
- $v(s)$ originally 0

$$v(a3) = 77$$

$$v(b3) = 48.6$$

$$\mathcal{R}_s = -3 \text{ and } \gamma = 1$$

$$\text{N: } -3 + [0.8 * 77 + 0.1 * (-100) + 0.1 * 0] = 48.6$$

$$\text{W: } -3 + [0.1 * 77 + 0.8 * 0 + 0.1 * 0] = 4.7$$

Finding the Optimal Policy

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

Finding the Optimal Policy

$$v(s) \leftarrow \mathcal{R}_s + \left(\max_a \gamma \sum_{s'} \mathcal{P}_{ss'}^a * v(s') \right)$$

$$\pi(s) = \arg \max_a \sum_{s'} \mathcal{P}_{ss'}^a * v(s')$$

Value Iteration Algorithm

Require: mdp , an MDP with states $\mathcal{S}$, actions $\mathcal{A}$ transition model $\mathcal{P}$, rewards $\mathcal{R}$, discount γ , ϵ , maximum error in state value function

```
1: function ValueIteration( $mdp, \epsilon$ )
2:    $v \leftarrow v' \leftarrow 0$  for each  $s \in \mathcal{S}$ 
3:    $\delta \leftarrow 0$  ▷ Maximum change in utility per iteration
4:   repeat
5:      $v \leftarrow v'; \delta \leftarrow 0$ 
6:     for each state  $s \in \mathcal{S}$  do
7:        $v[s] \leftarrow \mathcal{R}(s) + \gamma \max_{a \in \mathcal{A}(s)} \sum_{s'} \mathcal{P}(s' | s, a) v[s']$ 
8:       if  $|v'[s] - v[s]| > \delta$  then  $\delta \leftarrow |v'[s] - v[s]|$ 
9:   until  $\delta < \epsilon(1 - \gamma)/\gamma$ 
10:  return  $v$ 
```

Optimality

- In order to specify a utility/value function, we must answer several questions
 - Is there a finite or infinite horizon for decision making
 - Finite horizon: after N time steps have passed, no additional utility can be gained,
 $U_h([s_0, s_1, \dots, s_{N+k}]) = U_h([s_0, s_1, \dots, s_N])$
 - Given the grid world above, starting at (3,1) the agent **must** head for +1 if $N = 3$, so optimal decision is “Up”. With a longer horizon, the safe route can be taken instead
 - Given a finite horizon, the optimal action in a given state could change over time; such an optimal policy is **non-stationary**
 - If the optimal action depends only on the current state, then the optimal policy is **stationary**

Synchronous Dynamic Programming Algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Equation + Greedy Policy Improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

- Algorithms are based on state-value function $v_{\pi}(s)$ or $v_*(s)$
- Complexity $O(mn^2)$ per iteration, for m actions and n states
- Could also apply to action-value function $q_{\pi}(s, a)$ or $q_*(s, a)$
- Complexity $O(m^2n^2)$ per iteration

Outline

- 1 Recap
- 2 Introduction
- 3 Policy Evaluation
- 4 Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- 5 Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- 6 Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- 7 Contraction Mapping

Asynchronous Dynamic Programming

- DP methods described so far used **synchronous** backups
- i.e. all states are backed up in parallel
- **Asynchronous DP** backs up states individually, in any order
- For each selected state, apply the appropriate backup
- Can significantly reduce computation
- Guaranteed to converge if all states continue to be selected

Asynchronous Dynamic Programming

Three simple ideas for asynchronous dynamic programming:

- **In-place** dynamic programming
- Prioritised sweeping
- Real-time dynamic programming

In-Place Dynamic Programming

- Synchronous value iteration stores two copies of value function
for all $s \in S$

$$v_{new}(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in S} \mathcal{P}_{ss'}^a v_{old}(s') \right)$$

$$v_{old} \leftarrow v_{new}$$

- In-place value iteration only stores one copy of value function
for all $s \in S$

$$v(s) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in S} \mathcal{P}_{ss'}^a v(s') \right)$$

Prioritised Sweeping

- Use magnitude of Bellman error to guide state selection, e.g.

$$\left| \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v(s') \right) - v(s) \right|$$

- Backup the state with the largest remaining Bellman error
- Update Bellman error of affected states after each backup
- Requires knowledge of reverse dynamics (predecessor states)
- Can be implemented efficiently by maintaining a priority queue

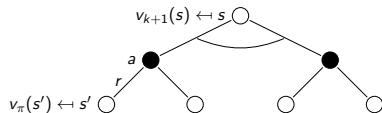
Real-Time Dynamic Programming

- Idea: only states that are relevant to agent
- Use agent's experience to guide the selection of states
- After each time-step $S_t, A_t, \mathcal{R}_{t+1}$
- Backup the state S_t

$$v(S_t) \leftarrow \max_{a \in \mathcal{A}} \left(\mathcal{R}_{S_t}^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{S_t s'}^a v(s') \right)$$

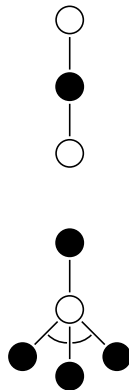
Full-Width Backups

- DP uses full-width backups
- For each backup (sync or async)
 - Every successor state and action is considered
 - Using knowledge of the MDP transitions and reward function
- DP is effective for medium-sized problems (millions of states)
- For large problems DP suffers Bellman's curse of dimensionality
 - Number of states $n = |\mathcal{S}|$ grows exponentially with number of state variables
- Even one backup can be too expensive



Sample Backups

- In subsequent lectures we will consider **sample backups**
- Using sample rewards and sample transitions $\langle S, A, R, S' \rangle$
- Instead of reward function $\mathcal{R}$ and transition dynamics $\mathcal{P}$
- Advantages:
 - Model-free: no advance knowledge of MDP required
 - Breaks the curse of dimensionality through sampling
 - Cost of backup is constant, independent of $n = |\mathcal{S}|$



Approximate Dynamic Programming

- Approximate the value function
- Using a **function approximator** $\hat{v}(s, \mathbf{w})$
- Apply dynamic programming to $\hat{v}(\cdot, \mathbf{w})$
- e.g. Fitted Value Iteration repeats at each iteration k ,
 - Sample states $\tilde{\mathcal{S}} \subseteq \mathcal{S}$
 - For each state $s \in \tilde{\mathcal{S}}$, estimate target value using Bellman optimality equation,

$$\tilde{v}_k(s) = \max_{a \in \mathcal{A}} \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \hat{v}(s', \mathbf{w}_k) \right)$$

- Train next value function $\hat{v}(\cdot, \mathbf{w}_{k+1})$ using targets $\{\langle s, \tilde{v}_k(s) \rangle\}$

Outline

- 1 Recap
- 2 Introduction
- 3 Policy Evaluation
- 4 Policy Iteration
 - Policy Improvement
 - Extensions to Policy Iteration
- 5 Value Iteration
 - Value Iteration in MDPs
 - Summary of DP Algorithms
- 6 Extensions to Dynamic Programming
 - Asynchronous Dynamic Programming
 - Full-Width and Sample Backups
- 7 Contraction Mapping

Some Technical Questions

- How do we know that value iteration converges to v_* ?
- Or that iterative policy evaluation converges to v_π ?
- And therefore that policy iteration converges to v_* ?
- Is the solution unique?
- How fast do these algorithms converge?
- These questions are resolved by **contraction mapping theorem**

Value Function Space

- Consider the vector space $\mathcal{V}$ over value functions
- There are $|\mathcal{S}|$ dimensions
- Each point in this space fully specifies a value function $v(s)$
- What does a Bellman backup do to points in this space?
- We will show that it brings value functions **closer**
- And therefore the backups must converge on a unique solution

Value Function ∞ -Norm

- We will measure distance between state-value functions u and v by the ∞ -norm
- i.e. the largest difference between state values,

$$\|u - v\|_{\infty} = \max_{s \in \mathcal{S}} |u(s) - v(s)|$$

Bellman Expectation Backup is a Contraction

- Define the Bellman expectation backup operator T^π ,

$$T^\pi(v) = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v$$

- This operator is a γ -contraction, i.e. it makes value functions closer by at least γ ,

$$\begin{aligned}\|T^\pi(u) - T^\pi(v)\|_\infty &= \|(\mathcal{R}^\pi + \gamma \mathcal{P}^\pi u) - (\mathcal{R}^\pi + \gamma \mathcal{P}^\pi v)\|_\infty \\ &= \|\gamma \mathcal{P}^\pi(u - v)\|_\infty \\ &\leq \|\gamma \mathcal{P}^\pi\| \|u - v\|_\infty \\ &\leq \gamma \|u - v\|_\infty\end{aligned}$$

Contraction Mapping Theorem

Theorem

For any metric space $\mathcal{V}$ that is complete (i.e. closed) under an operator $T(v)$, where T is a γ -contraction,

- T converges to a unique fixed point*
- At a linear convergence rate of γ*

Convergence of Iter. Policy Evaluation and Policy Iteration

- The Bellman expectation operator T^π has a unique fixed point
- v_π is a fixed point of T^π (by Bellman expectation equation)
- By contraction mapping theorem
- Iterative policy evaluation converges on v_π
- Policy iteration converges on v_*

Bellman Optimality Backup is a Contraction

- Define the Bellman optimality backup operator T^* ,

$$T^*(v) = \max_{a \in \mathcal{A}} \mathcal{R}^a + \gamma \mathcal{P}^a v$$

- This operator is a γ -contraction, i.e. it makes value functions closer by at least γ (similar to previous proof)

$$\|T^*(u) - T^*(v)\|_{\infty} \leq \gamma \|u - v\|_{\infty}$$

Convergence of Value Iteration

- The Bellman optimality operator T^* has a unique fixed point
- v_* is a fixed point of T^* (by Bellman optimality equation)
- By contraction mapping theorem
- Value iteration converges on v_*

Lecture Summary

- Stochastic Planning
- Dynamic Programming Solutions
 - Policy Iteration
 - Value Iteration
 - Extensions to DP
- Contraction Mapping